arXiv:2407.19985v2 [cs.CV] 30 Jul 2024

Mixture of Nested Experts: Adaptive Processing of Visual Tokens

Gagan Jain[◇⋆] **Nidhi Hegde**[◇] **Aditya Kusupati**[◇†]
Arsha Nagrani[◇] **Shyamal Buch**[◇] **Prateek Jain**[◇] **Anurag Arnab**[◇] **Sujoy Paul**[◇⋆]
[◇]Google DeepMind[◦] [†]University of Washington
{jaingagan,sujoyp}@google.com

Abstract

The visual medium (images and videos) naturally contains a large amount of information redundancy, thereby providing a great opportunity for leveraging efficiency in processing. While Vision Transformer (ViT) based models scale effectively to large data regimes, they fail to capitalize on this inherent redundancy, leading to higher computational costs. Mixture of Experts (MoE) networks demonstrate scalability while maintaining same inference-time costs, but they come with a larger parameter footprint. We present Mixture of Nested Experts (MoNE), which utilizes a nested structure for experts, wherein individual experts fall on an increasing compute-accuracy curve. Given a compute budget, MoNE learns to dynamically choose tokens in a priority order, and thus redundant tokens are processed through cheaper nested experts. Using this framework, we achieve equivalent performance as the baseline models, while reducing inference time compute by over *two-fold*. We validate our approach on standard image and video datasets - ImageNet-21K, Kinetics400, and Something-Something-v2. We further highlight MoNE's adaptability by showcasing its ability to maintain strong performance across different inference-time compute budgets on videos, using only a single trained model.

1 Introduction

Visual tokens, the fundamental building blocks of image and video representations, often exhibit strong inter-dependencies, spatially in images and spatio-temporally in videos. This offers a potential avenue for optimization in visual processing, as processing every token with equal emphasis may not be necessary for achieving optimal results. Traditional Vision Transformer (ViT) [18] and Video Vision Transformer (ViViT) [2] based models, however, process all tokens with equal emphasis, disregarding this inherent codependency and leading to unnecessary computational burden. This becomes a major bottleneck when deploying these models in real-world scenarios, where computational resources may be limited and real-time processing is required.

To this end, conditional computation has become a promising line of research to increase the capacity of a network, while only conditionally activating a part of it during inference. Sparse Mixture of Experts (MoEs) was initially popularized for Natural Language Processing (NLP) [38, 20],but it has been gaining attention for furthering conditional computation ideas in vision [35, 1, 31, 46] as well. While MoEs bring in improved performance at a given inference cost, they also increase the overall parameter count, leading to increased storage requirements. Moreover, these works rely on experts that have the same parameter count and compute, limiting their ability to reduce computational costs without resorting to skipping tokens entirely.

[◦]work done while at Google Research
[⋆]equal contribution

Preprint. Under review.

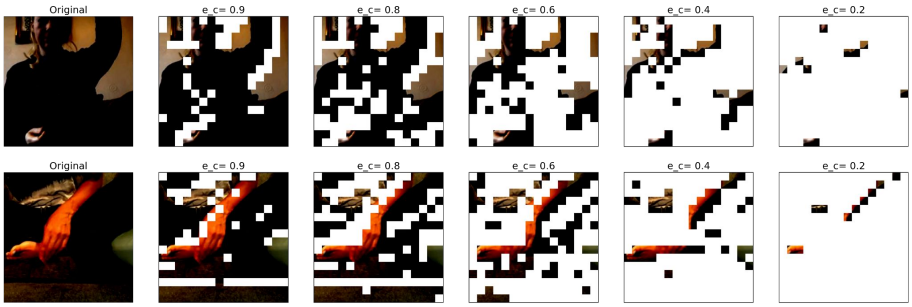


Figure 1: **MoNE's learned token importance:** From *left to right*, fewer image tokens are processed using the full model – to fit a compute budget – by an increasing threshold on MoNE's router logits.

In this work, we devise the **Mixture of Nested Experts (MoNE)** framework, which provides a scalable approach to conditional computation, bringing in significant reductions at inference time, while working with the same parameter space as the baseline model. MoNE draws inspiration from nested architectures [43, 28, 49], particularly MatFormer [17], that learns multiple representations of the same data with varying levels of details, based on structured slices of the parameter space. MoNE employs these structured nested models as experts in the MoE framework (without increasing parameter count), and learns a network to route tokens to these experts. We explore various design choices and present an effective recipe for allocating compute to experts, assigning tokens to experts, and training the MoNE framework. For the assignment operation, we propose Expert Preferred Routing (EPR), a routing algorithm that greedily assigns tokens to experts under capacity constraints based on router predictions. Figure 1 shows token importance as perceived by MoNE. We propose the following **three primary contributions**:

1. We introduce the novel Mixture of Nested Experts (MoNE) framework to dynamically allocate computational resources for Vision Transformer (ViT) based models.
2. Given a fixed parameter count, MoNE offers the flexibility of learning networks at much lower FLOPs ($\sim 2.3\times$ on video datasets), while still matching baseline performances.
3. Rigorous experiments show that MoNE works well for both image and video transformers, and visualizations depict that tokens routed to larger experts correlate well with regions of interest.

2 Related Work

Transformers [41] have become the de-facto architecture for processing data across multiple modalities spanning language [9, 32], images [18, 15], video [2, 45] and audio [21] and combinations thereof [34]. Consequently, there have been numerous efforts to improve the efficiency of transformers to make them more amenable for deployment in real-world applications [40]. These include approaches like efficient approximations of attention [11, 44], local attention [29, 3, 12] and reducing the number of tokens in the transformer [36, 27, 7] among others. Our work focuses on conditional computation [4], observing that some input tokens are easier to process than others, and therefore require less computation during inference.

Mixtures-of-Experts (MoE) transformers learn to route tokens to one of multiple expert MLPs [38, 20]. Although such models conditionally process input tokens, each expert has the same parameter- and FLOP-count, meaning that the total computation is constant for each input. More relevant to our approach, Mixture of Depths [33] extends the routing logic of MoE to conditionally skip an expert completely, thus total computation for each input varies dynamically. Completely skipping tokens being a hard unretrievable decision, our work chooses from an array of nested network, which effectively process information and help to stabilize training by getting rid of discontinuities.

Nested architectures [43, 28, 49] on the other hand, learn hierarchical representations of the input, where the first k hidden dimensions encode the most relevant information. This allows to extract multiple models with varying inference compute from a single trained model, similar to 'Mix-n-Match' in [17]. However, these models do not process tokens adaptively. Our model, in contrast, consists of a learned router which dynamically routes tokens to experts of different hidden dimensions based on the given compute constraints. Therefore, instead of requiring the user to select the hidden dimensions of

each transformer layer, our model only needs a single compute constraint input. Moreover, we show experimentally the superior accuracy-efficiency trade-offs achieved by our approach.

We note that other conditional computation approaches include "early exiting" [42, 37, 19, 26] such that the processing of "easy inputs" terminates before passing through all layers of the transformer. In addition, the ACT [23] algorithm was proposed for recurrent neural networks, and uses a "ponder cost" to learn a "halting score" for when to stop processing a particular input. This has since been extended to recurrent transformers [13], and also to each individual token in a transformer [48, 47], thus adaptively determining which tokens in a transformer to process. In contrast, our approach does not drop tokens, rather processes them with smaller nested models. This allows us to retain most of the information, and hence dampen the effect of irrecoverable decisions. We experimentally verify that our adaptive approach offers strong compute-performance trade-offs. Flextron [10] is a concurrent work, which looks at elastic inference, specified by user latency needs, with a focus on language modeling. Unlike Flextron, MoNE is guaranteed to learn models bounded by the specified latency needs and is able to learn from a single training phase, without using a surrogate model.

3 Preliminaries

Here, we discuss the concept of *nested models*, on which we build Mixture of Nested Experts (MoNE), followed by a discussion about Mixture of Nested Experts (MoE), and its differences from MoNE.

3.1 Nested Models

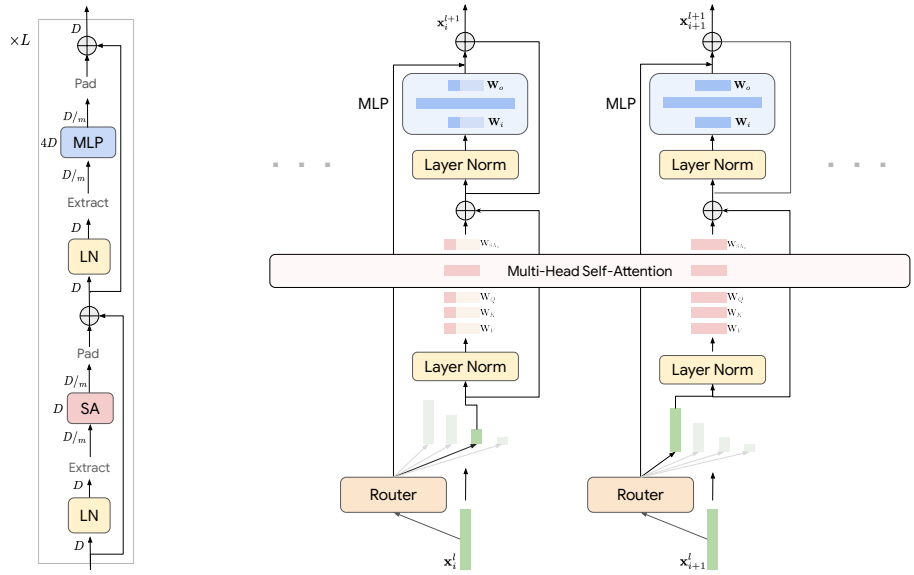
For the purposes of this work, we use the Vision Transformer (ViT) [18] as an example of a full model, from which nested submodels can be derived. Inspired by MatFormer [17], we define these submodels for every layer of the network, for both Self-Attention and MLP. The key idea is that in a feature projection operation $\mathbf{W}\mathbf{x}$, where $\mathbf{W} = \left[\mathbf{W}_{[:\frac{D}{m}]}, \mathbf{W}_{[\frac{D}{m}:]}\right]$, and $\mathbf{W}_{[:\frac{D}{m}]}$ denotes "slicing" the first $\frac{D}{m}$ dimensions, we can extract a partial projection $\mathbf{W}_{[:\frac{D}{m}]}\mathbf{x}_{[:\frac{D}{m}]}$. This can be done for any projection in the transformer, and we can extract smaller models from it. We refer to these as nested models, and D/m as the nested model dimension. This is shown in Figure 2a. The *Extract* operation extracts the first D/m features and applies the corresponding projection sub-matrix to it, while the *Pad* operation pads it back to full dimension D before residual connections and LayerNorm. While MatFormer applies the nested structure only to the hidden dimension of the MLP layer, in our approach we extend it to the in- and out-projections of both the Self-Attention (SA) and MLP layer. In the SA block, irrespective of the sub-model used in the in-projections, it is always projected to the model dimension D for the $(\mathrm{QK}^T)\mathrm{V}$ operation. The same thing is performed in MLP, where the hidden dimension is always $4D$, as in ViT, irrespective of the dimension of the in/out-projection.

We extract E nested models with exponentially-spaced model dimensions. Therefore, for a typical value of $E = 4$, the model dimension for the nested models are $\left[\frac{D}{8}, \frac{D}{4}, \frac{D}{2}, D\right]$. Note that while we build upon the idea of nested models from MatFormer, we do not share their training strategy which involves joint optimization through a weighted loss over these submodels. In contrast, we treat these nested models as distinct experts with varying compute requirements. The Mixture of Nested Experts (MoNE) framework (described in detail in Sec. 4.1) then dynamically routes input tokens to these nested experts based on their information content, with the idea that more informative tokens should be processed by larger (and thus more computationally expensive) nested models.

3.2 Mixture of Experts

A Mixture of Experts (MoE) layer in a transformer can be represented as $\mathrm{MoE}(\mathbf{x}) = \sum_{i=1}^{E} g(\mathbf{x})_i e_i(\mathbf{x})$, where E is the number of experts, $e_i()$ are the expert models each having their own parameters, $g : \mathbb{R}^D \rightarrow \mathbb{R}^E$ is the routing/gating function, which decides the experts which should process $\mathbf{x}$. Note that g is sparse with only $k << E$ non-zero terms. During inference, only those experts are active.

MoE strictly increases the parameter count, but maintains the same inference FLOPs by setting $k = 1$. However, it still needs to process all tokens with the same pre-defined compute. In contrast, in MoNE, we do not extend the parameter count of the model, due to the nesting structure (see Sec. 3.1), and dynamically choose a nested expert during inference. Unlike in MoE, where all experts have the same capacity, in MoNE, $e_i \subset e_{i+1}$ with $k = 1$, which allows us to dynamically allocate compute.



(a) (b)

Figure 2: (a) **Nested model:** Partial in- and out-projections in the SA and MLP layers create nested models. m controls the parameter count and the FLOPs of nested models. The self-attention information exchange happens at the full model dimension D, MLP dimension is set to $4D$ as in ViT. (b) **Mixture of Nested Experts (MoNE)**: Each token $\mathbf{x}$ is routed to a nested network, denoted by different model dimension in the diagram. Here $\mathbf{x}_i$ gets routed to a nested model with model dimension $D/4$, whereas $\mathbf{x}_{i+1}$ gets to the full model. The information exchange between these tokens of different dimension happens in the self-attention block, where they are always projected to the same dimension. The router weights are also multiplied with the features for proper flow of gradients. A lighter color in the weight matrix indicate a sliced matrix to construct the nestedness.

4 Methodology

In this section, we describe the details of our Mixture of Nested Experts (MoNE) framework for efficient inference. We assume a Vision Transformer (ViT) [18] based architecture for our approach, and then extend it to Video ViT (ViViT) [2] as well.

4.1 Mixture of Nested Experts (MoNE)

Tokenization: In this paper, as our primary focus is images and videos, the model input is in $\mathbb{R}^{H\times W\times 3\times T}$, where $T = 1$ for images and $T > 1$ for videos. After tokenization, the input to the transformer is $\mathbf{X} \in \mathbb{R}^{D\times N}$ where N is the number of tokens, and D their model dimension. For images, we have $N = H/p_h \cdot W/p_w$, and for video, $N = T/p_t \cdot H/p_h \cdot W/p_w$, where H, W, T are the input height, width and duration respectively. p_h, p_w and p_t are the patch sizes along these respective dimensions. We use the ViT [18] and ViViT [2] architectures to tokenize images and videos respectively, obtaining a list of tokens $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^{N}$.

MoNE Block: The Mixture of Nested Experts (MoNE) framework is a dynamic routing mechanism that processes visual tokens using nested models with varying computational capacities, instead of processing all tokens with the full model. A pictorial repsentation of the model is presented in Figure 2b. Let $\mathcal{B}^l = \{\mathcal{B}^l_1, \ldots, \mathcal{B}^l_E\}$ denote the nested blocks at a certain layer l with increasing parameter sizes, $\mathcal{B}^l_E(.)$ being the full model block. A router network decides the appropriate nested block to use for every token. Hence information from tokens of different model dimension interact with each other. This is enabled by performing self-attention at the full model dimension D as discussed before. For each token $\mathbf{x}_i$, a router produces a probability distribution over the E nested experts, $\mathbf{r}_i = \text{softmax}(\mathbf{W_r}\mathbf{x}_i + \mathbf{b_r})$, where $\mathbf{W_r}$ and $\mathbf{b_r}$ denote the router weights and bias respectively.

These router predictions are sent to an assignment algorithm, which assigns every token to a single appropriate nested expert. Based on the assignments, we update the features for the ith token in the lth layer as follows -

$$\mathbf{x}_i^{l+1} = \mathbf{z}_i^l + \left(\alpha \mathbf{r}_{i,j}^l + 1\right) \cdot \mathcal{B}_j^{\text{FFN},l}(\mathbf{z}_i^l) \qquad \mathbf{z}_i^l = \mathbf{x}_i^l + \mathcal{B}_j^{\text{SA},l}(\mathbf{x}_i^l) \tag{1}$$

where the jth nested expert is chosen by the Expert Preferred Router [EPR(.)] algorithm for the ith token as per Eq. 2:

$$j^* = \text{EPR}\big(i; \{\mathbf{r}_i^l\}_{i=1}^N\big) \tag{2}$$

Note that the multiplication of the router predictions with the model output in Eq. 1 allows gradient propagation through the router weights. We also introduce a learnable parameter $\alpha \in [0, 1)$, initialized to 0, which ensures proper gradient flow during the initial training stages, specifically during finetuning from a pre-trained MatFormer model. Without scaling, a low initial router prediction would dampen the block output, whereas the initial multiplicative factor being 1 ensures a stable starting point.

Features and Loss: The feature of the last layer $\mathbf{x}_i^L$ is used for downstream applications. For classification tasks, we apply global average pooling on all the token features and apply a linear classifier layer to predict the categories.

4.2 Token to Nested Expert Assignments

Within the MoNE framework, the routing strategy is crucial for achieving an optimal balance between performance and computational efficiency. Traditionally there are two primary routing strategies – token choice [38] and expert choice [35] . In token-choice routing, the router predicts the probability distribution over the available experts, and picks the expert with the highest probability. However, this can suffer from load balancing issues, with most of the tokens being routed to one or few experts. Hence, inference time compute is only bounded by the compute of the full model. On the other hand, in expert choice routing, each expert selects the top-k tokens with the highest preference for that expert. This guarantees perfect bounds on computation. Potential conflicts due to token selection by multiple experts are resolved by prioritizing based on model size.

Formally, we consider a given distribution of nested models applied to the tokens, represented as $\mathbf{c} = \{c_1, \ldots, c_E\}$, s.t., $\sum_i c_i = 1$, which we call the capacity distribution over the nested models. The method for obtaining a suitable capacity distribution, given the inference time compute requirements, will be discussed in Sec. 4.3. Given router probabilities $\mathbf{r}_i$ for N tokens across E experts, we employ an Expert Preferred Routing algorithm (Algorithm 1). This is a greedy assignment approach that gives higher preference to larger nested models, aiming to identify the most important tokens first. We begin by examining the router predictions for the biggest to the smallest model, assigning $k_j = \lfloor c_j N \rfloor$ of the remaining tokens to j^{th} nested model. Any remaining tokens, arising from integer packing constraints, are assigned to the smallest model. Algorithm 1 presents the proposed Expert Preferred Routing (EPR) algorithm.

Algorithm 1 Expert Preferred Routing (EPR)

Require: $\mathbf{r} \in \mathbb{R}^{E \times N}$ (router predictions), $\mathbf{c}$ (capacity distribution, s.t., $\mathbf{c}^T\mathbf{1} = 1$),
Ensure: $M \in \{1, \ldots, E\}^N$ (nested model index)

```
M ← 1_T                                   Default assignments to the smallest model
for j = E to 1 do
    k_j ← ⌊c_j · T⌋
    I ← Top-k-Index(r[j, ...], k_i)       Returns value and indices of Top-K
    M[I] ← j
    r[:, I] ← 0                           Null out assigned ones
end for
return M
```

4.3 Capacity Distribution Across Experts

The Expert Preferred Routing (EPR) as described in Section 4.2 needs the individual expert's capacity bounds c_i to be specified. To get this, we define a metric called the effective capacity : $e_c = \sum_{i=1}^{E} c_i d_i / D$, where $d_i = D/2^{E-i}$ is the model dimension of the ith nested model. Given a certain inference FLOP requirement, we can translate that to an equivalent effective capacity e_c. Since every token gets processed through exactly one nested expert, this along with the given budget imposes two constraints on the unknown capacity distribution $\mathbf{c}$. However, since the individual expert capacities vary log-linearly, multiple distributions $\mathbf{c}$ can lead to the same e_c for $E > 2$ and it is non-trivial to choose

one over the other. MoEs generally use auxilliary loss functions [35, 38] to promote equal usage of experts. But in MoNE, that would render a certain fixed capacity, missing out on the flexibility that the framework provides to function with any capacity. Hence, we invoke intuitive constraints to solve for $\mathbf{c}$. Specifically, we incentivize the usage of larger models, while also adding an entropy term to ensure uniformity of capacities across experts. Given these constraints, we solve the following optimization problem:

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^{E} \frac{c_i}{\delta^{i-1}} - \beta \sum_{i=1}^{E} c_i \cdot \log c_i \\ \text{subject to} \quad & \sum_{i=1}^{E} c_i = 1 \qquad \sum_{i=1}^{E} \frac{c_i}{2^{E-i}} = e_c \qquad 0 \leq c_i \leq 1 \quad \forall i \in \{1, ..., E\} \\ \text{given} \quad & 0 < e_c < 1, \quad E, \delta > 1, \quad \beta > 0 \end{aligned} \tag{3}$$

In practice, we set (β, δ) to $(10, 2)$ and use a Sequential Least SQuares Programming (SLSQP) optimizer to solve Eq. 3 for the capacity distribution $\mathbf{c}$, which is then used by EPR (Algorithm 1) to get token to expert mappings. We empirically verify these choices in Section 6.

4.4 Videos

MoNE can be seamlessly adapted for video-based tasks. In videos, there exists another dimension – time – which adds to the significant redundancy in the tokens. Given the large number of tokens that can be obtained from a video, the computational costs grow drastically. To tackle this problem, works in literature factorize computation along space and time [2, 5], perform local windowed computation [30], etc. MoNE being a token based approach, directly extends to video encoders.

For video processing, we leverage the Factorized Encoder architecture of ViViT [2]. This architecture employs two distinct transformers: spatial and temporal. After tokenization, each temporal index yields a set of tokens representing information from local spatio-temporal neighborhoods. These spatial tokens interact within their temporal index for L_s layers, culminating in a single global token per index. Subsequently, a temporal transformer processes these global tokens across L_t layers. Given that the spatial transformer significantly dominates computational costs in this model, we integrate MoNE into the spatial component while maintaining full capacity for the temporal transformer. The router predicts expert assignments for all temporal frames independently, which are then consumed by the EPR(.) algorithm to produce frame-wise expert assignments.

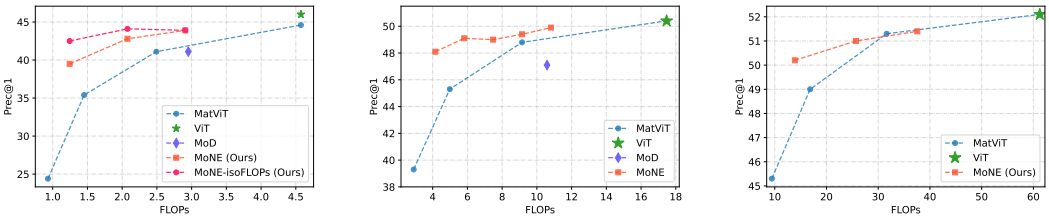
5 Results

In this section, we empirically evaluate MoNE on multiple datasets spanning images and videos, for different sizes and assess its adaptability to stringent FLOP constraints during inference.

Implementation details: We empirically evaluate MoNE on image and video classification. For image classification, we train the network with random initialization. As for video classification, we follow previous literature and start from a pre-trained MatViT [17] model due to the inherent nested structure required in MoNE. We follow the joint training strategy of MatViT, with separate losses an all model granularities. We implement MoNE on JAX [8] using BigVision [6] for image classification and Scenic [14] for video classification. We follow the AugReg [39] training strategy to train all our image classification models. For video classification tasks, we inherit all augmentations and hyperparameter values directly from the ViViT [2] paper.

For all experiments in this section, we place a single router at the first transformer layer, and propagate the router decisions to all the layers. We also multiply the router predictions (Eqn 1) to all layers, which ensures differentiable paths through the router network in all layers and allows the more evolved features from later layers to influence router learning. We also perform analysis of router placement in Section 6.

Baselines: We first compare with MatViT's nested models. As mentioned in the paper [17], we perform joint training over all four nested models that we consider in this work - $\{\frac{D}{8}, \frac{D}{4}, \frac{D}{2}, D\}$. MatViT is equivalent to MoNE, with a deterministic router to pass all tokens to the same nested model. We show that adaptively mixing tokens with different model dimensions performs much



(a) S/16 (b) B/16 (c) L/16

Figure 3: **Image classification:** Performance comparison of MoNE with baselines on ImageNet-21k for different model sizes. MoNE performs significantly better than MatViT and Mixture-of-Depth (MoD) and even benefits from isoFLOPs training (see fig a).

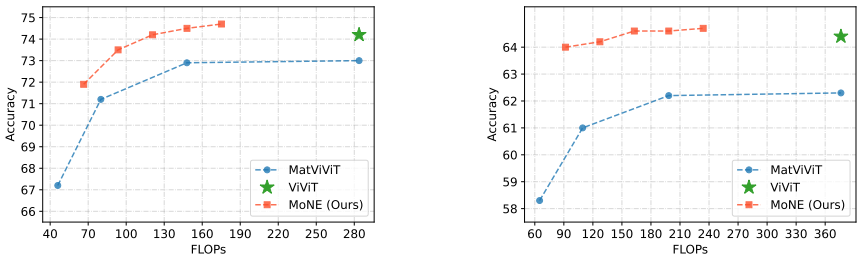
better across datasets and tasks. We also compare with Mixture of Depths (MoD) [33], which is also a token routing algorithm, but proposed for language tasks. MoD takes the extreme decision of either processing or skipping for every token in a layer. MoNE, on the other hand, makes fuzzy decisions to choose intermediate-sized models, instead of skipping, which helps to retain significant information at the expense of low compute. We adopt the best reported MoD configuration: processing 12.5% of tokens every other layer while processing all tokens in the remaining layers.

Images: First, we evaluate MoNE on ImageNet-21k [16] classification using ViT. We experiment with S, B, and L models to showcase the efficacy of MoNE across model sizes. As ImageNet-21k can have multiple labels for an image, we report the commonly used precision@1 metric. Figure 3 shows the results for all the models on ImageNet-21k. MoNE performs much better than MatViT's nested models and MoD, specifically in the low FLOPs regimes. MoNE achieves comparable performance to baselines with around $2\times$ reduction in FLOPs.

Following the literature on language models [33, 25], we experimented with isoFLOPs training, which involves training for the same number of FLOPs as the baseline models. Since MoNE models have fewer FLOPs compared to their ViT counterparts, they require more training epochs to achieve the same total training FLOPs. We conducted this experiment on the S/16 model (see Figure 3a) and observed additional improvements in MoNE's performance, particularly for the lower FLOPs models.

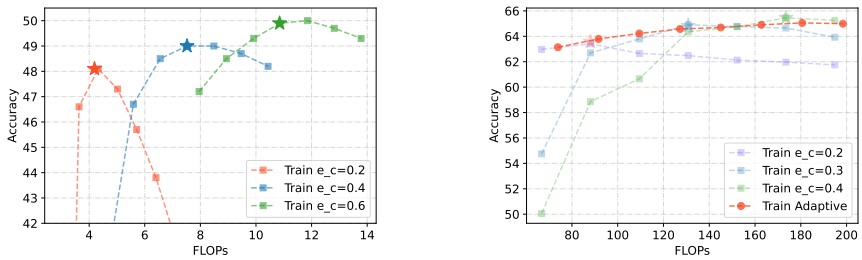
Videos: Since video models rely on heavy pre-training [2], we first train a baseline model with nested structure on the benchmark datasets - Kinetics-400 [28] and Something-Something-v2 (SSv2) [22]. We use the ViViT Factorized Encoder B/16 model [2] for our experiments and consistently report the 8x1 test accuracy, averaging predictions over 8 temporal clips [2]. Figure 4 illustrates the results of the MoNE framework, significantly outperforming the individual nested models. MoNE offers $2-3\times$ reduction in FLOPs compared to the ViViT baseline, without any accuracy drop (On SSv2, the FLOPs for MoNE are 162.8 vs 376.3, with similar accuracy – 64.6 vs 64.4). We always do isoFLOPs training while fine-tuning these models. We attribute the higher compute gains compared to images due to the greater (spatial and temporal) redundancy in videos, which MoNE exploits well.

Inference time capacity adaptation: Capacity adaptation during inference is crucial, as the inference time budget is often dynamic, changing based on user needs. Ideally, a model should adjust with little



(a) Kinetics-400 (b) Something-Something-v2

Figure 4: **Video classification**: MoNE vs. baselines on video datasets. Finetuning with the isoFLOPs training regime leads to matching baseline with $> 2\times$ FLOP improvement.



(a) ImageNet21k (b) Something-Something-v2

Figure 5: **Capacity adaptation during inference:** Performance changes when a model trained at a certain capacity (denoted as ⋆) is evaluated at other capacities. The "Train Adaptive" plot for SSv2 denotes a single model evaluated at different inference-time budgets.

to no retraining. To evaluate this ability, we test how MoNE, trained at a specific effective capacity (e_c) performs when evaluated at other capacities. Fig. 5 presents the results for image and video classification. We observe that the model adapts well to nearby capacities. However, as expected, its ability declines with extreme shifts in the capacity budget between train and eval. The performance degradation is steeper while adapting a model trained at high capacity to low capacity. We also note that the performance degrades more gracefully in videos than on images, presumably due to the larger temporal redundancy.

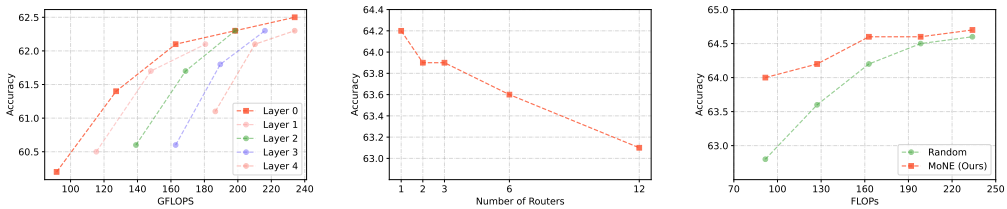
To enhance model adaptability, we train a model with the capacity sampled uniformly at random from $\{0.15, 0.25, \ldots, 0.95\}$ at each training step. The results on SS-v2 (Figure 5b) demonstrate our framework's strong capability to adapt to any inference-time budget using a single model. It is interesting to note that the training FLOPs of this adaptively trained model are equal to those of a baseline model (isoFLOPs training). The model adapts extremely well even to capacities that are significantly different ($\{0.2, 0.3, \ldots\}$) from those sampled during training.

6 Router Analysis

In this section, we discuss, analyse and visualise the design choices in implementing the router network. We choose the SSv2 dataset for this analysis.

Router Position: As discussed before, we use a single router at the first layer, and propagate its decisions for all layers. While a delayed router might benefit from a more processed feature representation as input, this also diminishes the compute gains, as the initial layers operate at full capacity. We reason this choice by monitoring performance while placing the router at different layers in the network. As Figure 6a suggests, the gains through richer features from the later layers is outweighed by the shift in the curve to the right, and an equivalent capacity with our default router produces higher points on the curve.

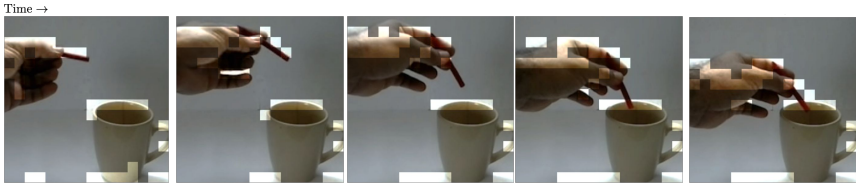
Number of Routers: We vary the number of routers, placing them at different regular intervals in the network in Figure 6b. The decision from one router is carried out until the next router block is encountered. We notice a clear downtrend in performance as we increase the number of routers from being present in the first layer to being present in all layers. Intuitively, more routers demands learning more decisions, and the main network in turn has to adapt to these decisions making it harder to optimize.



(a) Router Position (b) Number of Routers (c) Comparing with random router

Figure 6: **Router Analysis:** Effect of router placement and learning on Something-Something v2.

(a) Images from ImageNet-21k



(b) Video frames from SomethingSomethingv2

Figure 7: **Tokens routed to the full model:** Highlighted regions are the tokens sent to the full model, while rest of the tokens are sent to the smaller nested models. (a) shows examples on images and (b) shows an example on a video at multiple temporal indices. As we can see, the necessary and important tokens are sent to the full model.

Comparison with Random Router: We compare our learned router approach to a random router, which maps tokens to nested experts randomly, while still maintaining the capacity limits of each expert (c_i), as computed in Section 4.3. Results in Figure 6c suggests that with lower effective capacities, the random router performance degrades while the learned router still manages to understand relevant patterns from the input, thus upholding performance.

Visualizing Important Tokens: The above claim is further backed by visualizing the token importance during inference at a low effective capacity (e_c). We highlight the tokens selected by the largest expert, i.e., the full model on a few images in Figure 7a. It can be easily observed that the tokens sent to the largest model correlate well with the regions of interest in the images. On videos (Figure 7b) as well, the highlighted regions across temporal stamps consistently track the regions of motion.

Capacity Allotment: Given a fixed input capacity e_c, we demonstrate the superior performance of our heuristic-based allocation method (Section 4.3) compared to other approaches, as shown in Table 1. While the Proportionate allocation (assigning capacity inversely proportional to expert compute cost) and Uniform allocation (assigning equal capacity to all experts) show promising results, they lack the flexibility to adapt to varying budgets. Additionally, greedy approaches, such as allocating the entire budget to the largest expert and dropping other tokens (MoD style), or a greedy approach where the largest expert is assigned capacity such that all the remaining tokens are routed through the smallest expert, exhibit inferior performance.

Table 1: SSv2 Performance of different capacity distribution methods

	Static budget		Dynamic budget			
Distribution	Proportionate	Uniform	MoD Greedy [33]	Greedy	MoNE	MoNE
Effective Capacity (e_c)	0.27	0.47	0.4	0.4	0.3	0.4
Accuracy	64.3	64.6	63.9	64.2	64.2	**64.6**

7 Conclusion

In this work, we presented Mixture of Nested Experts (MoNE), a novel framework for adaptive processing of visual tokens by dynamically allocating computational resources to different tokens. Through a nested structure with shared parameters and the proposed expert-choice routing algorithm, MoNE achieves significant reductions in inference time (over two-fold) without sacrificing accuracy on benchmark image and video datasets. Future works can be centered around extending MoNE to denser tasks like object detection, captioning, etc.

Limitations: Extending this to auto-regressive decoding in LLMs is non-trivial, as this is designed primarily with an encoder architecture in mind. We leave this further exploration for future work.
Societal Impact: The MoNE framework dynamically allocates computational resources with a given budget, thereby significantly minimizing energy usage and carbon emissions during inference of

vision models. MoNE can also play a role in democratization of AI, allowing broader access to trained models without the need for large resources.

References

[1] J. U. Allingham, F. Wenzel, Z. E. Mariet, B. Mustafa, J. Puigcerver, N. Houlsby, G. Jerfel, V. Fortuin, B. Lakshminarayanan, J. Snoek, et al. Sparse moes meet efficient ensembles. *arXiv preprint arXiv:2110.03360*, 2021.

[2] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid. Vivit: A video vision transformer. In *ICCV*, pages 6836–6846, 2021.

[3] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. In *arXiv preprint arXiv:2004.05150*, 2020.

[4] Y. Bengio. Deep learning of representations: Looking forward. In *International Conference on Statistical Language and Speech Processing*, 2013.

[5] G. Bertasius, H. Wang, and L. Torresani. Is space-time attention all you need for video understanding? In *ICML*, volume 2, page 4, 2021.

[6] L. Beyer, X. Zhai, and A. Kolesnikov. Big vision. https://github.com/google-research/big_vision, 2022.

[7] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman. Token merging: Your vit but faster. In *ICLR*, 2022.

[8] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax.

[9] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *NeurIPS*, 2020.

[10] R. Cai, S. Muralidharan, G. Heinrich, H. Yin, Z. Wang, J. Kautz, and P. Molchanov. Flextron: Many-in-one flexible large language model. *ICML*, 2024.

[11] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, et al. Rethinking attention with performers. In *ICLR*, 2021.

[12] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *ACL*, 2019.

[13] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal transformers. In *ICLR*, 2019.

[14] M. Dehghani, A. Gritsenko, A. Arnab, M. Minderer, and Y. Tay. Scenic: A jax library for computer vision research and beyond. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21393–21398, 2022.

[15] M. Dehghani, J. Djolonga, B. Mustafa, P. Padlewski, J. Heek, J. Gilmer, A. P. Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *ICML*, 2023.

[16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.

[17] Devvrit, S. Kudugunta, A. Kusupati, T. Dettmers, K. Chen, I. Dhillon, Y. Tsvetkov, H. Hajishirzi, S. Kakade, A. Farhadi, P. Jain, et al. Matformer: Nested transformer for elastic inference. *arXiv preprint arXiv:2310.07707*, 2023.

[18] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. 2021.

[19] M. Elbayad, J. Gu, E. Grave, and M. Auli. Depth-adaptive transformer. In *arXiv preprint arXiv:1910.10073*, 2019.

[20] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[21] Y. Gong, Y.-A. Chung, and J. Glass. Ast: Audio spectrogram transformer. In *arXiv preprint arXiv:2104.01778*, 2021.

[22] R. Goyal, S. Ebrahimi Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Fruend, P. Yianilos, M. Mueller-Freitag, et al. The" something something" video database for learning and evaluating visual common sense. In *Proceedings of the IEEE international conference on computer vision*, pages 5842–5850, 2017.

[23] A. Graves. Adaptive computation time for recurrent neural networks. In *arXiv preprint arXiv:1603.08983*, 2016.

[24] D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus). In *arXiv preprint arXiv:1606.08415*, 2016.

[25] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[26] G. Huang, D. Chen, T. Li, F. Wu, L. Van Der Maaten, and K. Q. Weinberger. Multi-scale dense networks for resource efficient image classification. In *ICLR*, 2018.

[27] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira. Perceiver: General perception with iterative attention. In *ICML*, 2021.

[28] E. Kim, C. Ahn, and S. Oh. Nestednet: Learning nested sparse structures in deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8669–8678, 2018.

[29] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[30] Z. Liu, J. Ning, Y. Cao, Y. Wei, Z. Zhang, S. Lin, and H. Hu. Video swin transformer. In *CVPR*, pages 3202–3211, 2022.

[31] Y. Lou, F. Xue, Z. Zheng, and Y. You. Sparse-mlp: A fully-mlp architecture with conditional computation. *arXiv preprint arXiv:2109.02008*, 1:12, 2021.

[32] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 2020.

[33] D. Raposo, S. Ritter, B. Richards, T. Lillicrap, P. C. Humphreys, and A. Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[34] M. Reid, N. Savinov, D. Teplyashin, D. Lepikhin, T. Lillicrap, J.-b. Alayrac, R. Soricut, A. Lazaridou, O. Firat, J. Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. In *arXiv preprint arXiv:2403.05530*, 2024.

[35] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby. Scaling vision with sparse mixture of experts. *NeurIPS*, 34:8583–8595, 2021.

[36] M. S. Ryoo, A. Piergiovanni, A. Arnab, M. Dehghani, and A. Angelova. Tokenlearner: What can 8 learned tokens do for images and videos? In *NeurIPS*, 2021.

[37] T. Schuster, A. Fisch, J. Gupta, M. Dehghani, D. Bahri, V. Tran, Y. Tay, and D. Metzler. Confident adaptive language modeling. In *NeurIPS*, 2022.

[38] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.

[39] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer. How to train your vit? data, augmentation, and regularization in vision transformers. *arXiv preprint arXiv:2106.10270*, 2021.

[40] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 2022.

[41] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[42] A. Veit and S. Belongie. Convolutional networks with adaptive inference graphs. In *ECCV*, 2018.

[43] C. Wan, H. Hoffmann, S. Lu, and M. Maire. Orthogonalized sgd and nested architectures for anytime neural networks. In *International Conference on Machine Learning*, pages 9807–9817. PMLR, 2020.

[44] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity. In *arXiv preprint arXiv:2006.04768*, 2020.

[45] Y. Wang, K. Li, X. Li, J. Yu, Y. He, G. Chen, B. Pei, R. Zheng, J. Xu, Z. Wang, et al. Internvideo2: Scaling video foundation models for multimodal video understanding. In *arXiv preprint arXiv:2403.15377*, 2024.

[46] F. Xue, Z. Shi, F. Wei, Y. Lou, Y. Liu, and Y. You. Go wider instead of deeper. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8779–8787, 2022.

[47] F. Xue, V. Likhosherstov, A. Arnab, N. Houlsby, M. Dehghani, and Y. You. Adaptive computation with elastic input sequence. In *ICML*, 2023.

[48] H. Yin, A. Vahdat, J. M. Alvarez, A. Mallya, J. Kautz, and P. Molchanov. A-vit: Adaptive tokens for efficient vision transformer. In *CVPR*, 2022.

[49] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang. Slimmable neural networks. *arXiv preprint arXiv:1812.08928*, 2018.

A Appendix

A.1 MatFormer Structure on Model Dimension

Following MatFormer convention, we define E ViT blocks $\mathcal{B}_i$, such that $\mathcal{B}_i \subset \mathcal{B}_{i+1}$ for all $i \in [E]$, meaning that the parameters of $\mathcal{B}_i$ are contained in those of $\mathcal{B}_{i+1}$. With d_i denoting the hidden dimension corresponding to nested model $\mathcal{B}_i$ such that $d_1 < d_2 < \ldots d_E = D$, the block operation for a nesting $\mathcal{B}_i$ on an input token set $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^N$ for $\mathbf{x}_i \in \mathbb{R}^D$ is given by:

$$\mathcal{B}_i(\mathbf{X}) \triangleq \mathcal{B}(\mathbf{x}, \mathbf{d_i}) = \mathcal{B}^{\text{FFN}}(\mathbf{Z}, \mathbf{d_i}), \quad \mathbf{Z} = \mathcal{B}^{\text{SA}}(\mathbf{X}, \mathbf{d_i}) + \mathbf{X}, \qquad \mathbf{d_i} = \underbrace{(d_i, d_i, \ldots, d_i)}_{N \text{ times}} \tag{4}$$

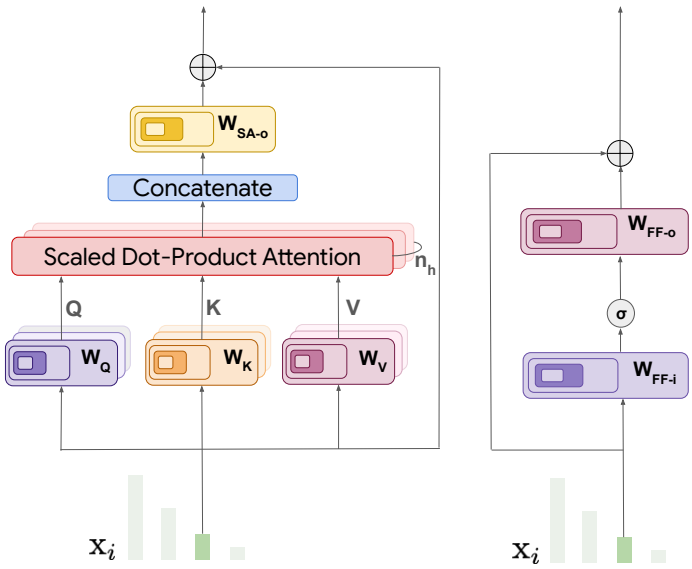


Figure 8: MatFormer Structure on Model Dimension

The modified Self-Attention $\mathcal{B}^{\text{SA}}$ and Feed-Forward $\mathcal{B}^{\text{FFN}}$ subroutines are shown below.

$$\mathcal{B}^{\text{SA}}(\mathbf{X}, \mathbf{d}) = \text{LN}\left[\left(\sigma\left(\frac{(\mathbf{X} \odot_{\mathbf{d}} \mathbf{W_Q}) \cdot (\mathbf{X} \odot_{\mathbf{d}} \mathbf{W_K})^T}{\sqrt{d_m}}\right)(\mathbf{X} \odot_{\mathbf{d}} \mathbf{W_V})\right) \boxdot_{\mathbf{d}} \mathbf{W_{SA_o}}\right] \tag{5}$$

$$\mathcal{B}^{\text{FFN}}(\mathbf{X}, \mathbf{d}) = \text{LN}\left[\sigma(\mathbf{X} \odot_{\mathbf{d}} \mathbf{W_{FF_i}}) \boxdot_{\mathbf{d}} \mathbf{W_{FF_o}}\right] \tag{6}$$

where $\odot_{\mathbf{d}}$ and $\boxdot_{\mathbf{d}}$ respectively denote the sliced in and out projection operators, such that:

$$(\mathbf{X} \odot_{\mathbf{d}} \mathbf{W})_j = (\mathbf{x}_j)_{[:d^j]} \cdot \mathbf{W}_{[:d^j]} \qquad (\mathbf{X} \boxdot_{\mathbf{d}} \mathbf{W})_j = \mathbf{x}_j \cdot (\mathbf{W}_{[:d^j]})^T \tag{7}$$

In the general Mixture of Nested Experts (MoNE) setting discussed in Section 4.1, the overall block computation for the set of tokens $\mathbf{X}$ requires knowledge of the expert assignments for each token beforehand. Given these assignments $\mathbf{m} \in \mathbb{R}^N$, such that $m_i \in \{1, 2, \ldots, E\}$, the computation for the ith token processed by the jth expert can be represented as:

$$\mathcal{B}_j(\mathbf{x}_i) \triangleq [\mathcal{B}(\mathbf{X}, \mathbf{d})]_i, \qquad \mathbf{d} = (d_{m_i})_{i=1}^N \tag{8}$$

In Eq. 8, the block update for token $\mathbf{x}_i$ is dependent on the complete input set $\mathbf{X}$ and their respective expert assignments $\mathbf{m}$, but we omit these in the definition $\mathcal{B}_j$ for notational convenience. Additionally,

this definition directly extends to the sub-routines $\mathcal{B}^{\text{SA}}$ and $\mathcal{B}^{\text{FFN}}$, as presented in Eq. 1. Here, the weight matrices of SA are $\mathbf{W_Q}, \mathbf{W_K}, \mathbf{W_V}, \mathbf{W_{SA_o}} \in \mathbb{R}^{D\times((D/n_h)\times n_h)}$ and the weight matrices of FFN are $\mathbf{W_{FF_i}}, \mathbf{W_{FF_o}} \in \mathbb{R}^{D\times d_{ff}}$, ignoring bias terms for simplicity. $\mathbf{W}_{[:k]}$ denotes the first k rows of $\mathbf{W}$. Here, n_h denote the number of heads in the attention mechanism, d_{ff} denotes the feed forward dimension, and σ denotes a non-linearity, typically set to GeLU [24].